ChainWork

Decentralised Freelance Platform

Team LedgerLogic

Team LedgerLogic

Devanshi Mahto	Shambhavi S Vijay
Priyanshi Mahto	Vaishnavi Ventrapragada
Sudhiksha Vijayagiri	Usepetla Nancy Sahithi

Decentralization: A Better Approach

Challenges in Centralized Freelance Platforms

- Intermediary Dependence
- Payment Delays
- High Fees
- Manual Disputes
- Limited Transparency

The Alternative

- Decentralized Freelancing Model
- Escrow-Based Payment System
- On-Chain Reputation System
- Trustless Transactions using Smart Contracts

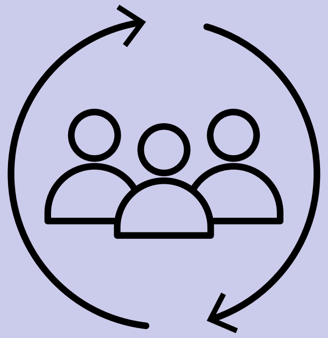
core features:

Smart Contracts

Automates hiring, payments, and job completion securely

Escrow Payment System

Funds are locked safely until work is completed

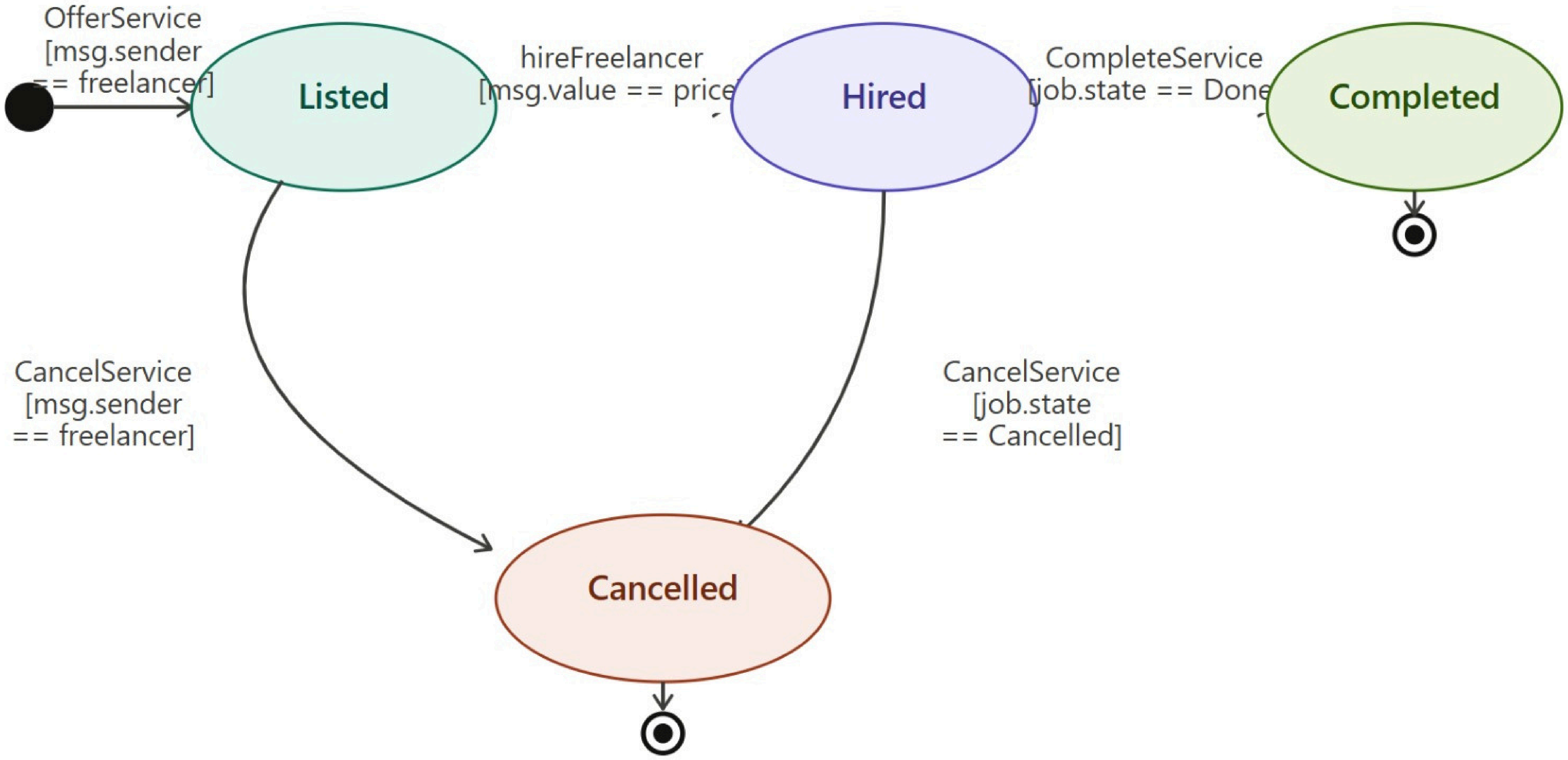


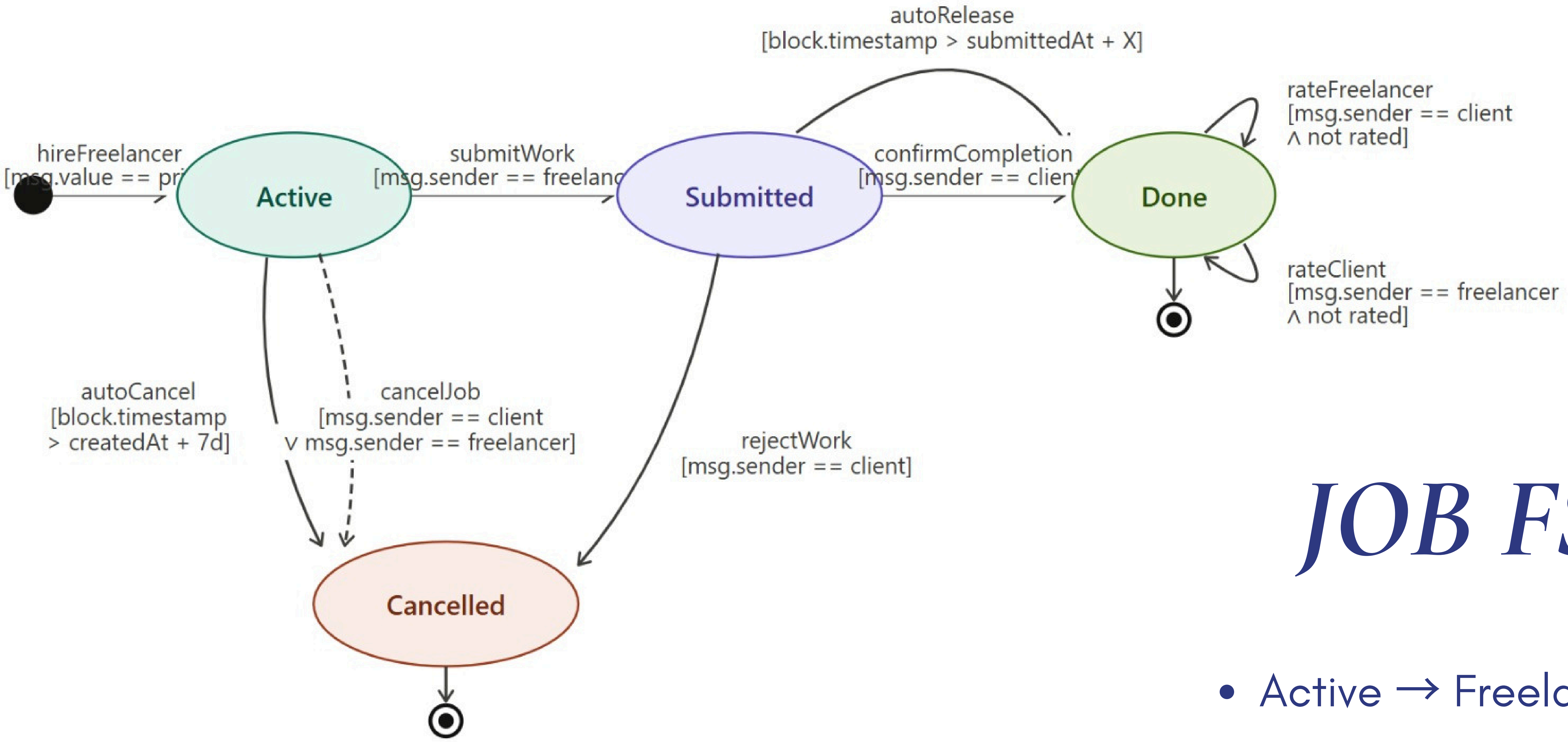
Reputation System

Freelancers are rated (1-5) and scores stored on-chain

Service FSM

- Listed → Service available for hiring
- Hired → Client selects service & locks payment
- Completed → Work finished successfully
- Cancelled → Service stopped before completion





JOB FSM

- Active → Freelancer is working
- Submitted → Work delivered
- Done → Client approves & payment released
- Cancelled → Job terminated

On Chain and Off Chain Storage

On Chain:

- Wallet addresses (freelancer, client)
- ETH amounts and escrow balances
- Job status (Active / Submitted / Done / Cancelled)
- Deadline and timestamps
- File hashes — metadataCid, workCid, jobDescription
- Reputation scores and weights
- Feedback tokens and stake balance

Off Chain:

- Service title, description, images, tags
- Full job brief text
- Deliverable files submitted by the freelancer

Client-Side Workflow

1. Stake Initialization

depositStake()

- Client invokes depositStake() to register participation
- Requires a minimum stake of 0.05 ETH
- Enforces Sybil resistance via economic barrier
- Mandatory prerequisite for accessing feedback functions
- Stake is non-slashable and remains locked as commitment
- Acts as identity verification through capital bonding

2. Job Creation & Escrow Locking

hireFreelancer(serviceId, deadline, jobDescription):

- Client invokes hireFreelancer(...) to instantiate a job contract
- Must transfer exact service price (msg.value == priceWei)
- Enforces constraint: caller ≠ service owner

Parameters:

- serviceId → reference to on-chain service struct
- deadline → bounded within [1, 365] days
- jobDescription → fixed-size bytes32 metadata field

State Transition:

- Job state initialized to Active
- Funds transferred to contract and held in escrow (contract balance)

3. Job Finalization

confirmCompletion(jobId):

- Client invokes confirmCompletion(jobId) to finalize job

Execution Semantics::

- Validates caller authorization (only client)
- Applies Checks-Effects-Interactions (CEI) patternState updated before external transfer
- Transfers escrowed ETH to freelancer

Post-State::

- Job state transitions to Completed
- Submitted data becomes immutable (on-chain persistence)
- Mints dual feedback tokens for both participants

4.Cancellation Logic & Edge Cases

cancelJob(jobId):

- Invoked to terminate job prior to completion

State-Based Access Control:

Active State:

- Callable by:

1. Client
2. Freelancer
3. Any address after deadline expiry

Submitted State:

- Before deadline → client-only access
- After deadline → permissionless (any caller)

Execution Outcome:

- Escrow funds refunded to client
- Job state transitions to Cancelled
- No state mutation in reputation mappings

5. Feedback Submission

submitFeedback(tokenId, score):

- Client invokes submitFeedback(...) using assigned tokenId
- Score constrained to uint8 range [1–5]

Preconditions:

- Token must be valid and unused
- Associated job must be Completed

Effect:

- Records rating in reputation aggregation logic
- Contributes to weighted scoring mechanism

Freelancer Workflow

1. Stake & Service Listing

Stake Deposit (depositStake)

- Minimum 0.05 ETH required
- Acts as Sybil resistance (prevents fake users)
- Required to participate in feedback/reputation system

Service Listing (offerService)

- Off-chain via IPFS: description, portfolio, files
- On-chain: freelancer address, price, metadata CID
- Service becomes visible to clients (Listed state)

2.Hiring & Escrow

Client Hiring (hireFreelancer):

- Client selects service & sends exact payment
- Cannot hire own listing (security check)

Escrow Mechanism:

- Funds locked using Smart Contracts
- Job created with Active status
- Deadline set for completion

Benefit:

- Guarantees payment availability before work starts

3.Work Submission

Work Upload:

- Freelancer uploads work to IPFS
- Generates Content Identifier (CID)

On-Chain Record (submitWork):

- CID stored on blockchain
- Job status → Submitted
- Work becomes tamper-proof & verifiable

Client Interaction:

- Client reviews submitted work before approval

4.Payment Resolution

Successful Completion:

- Client calls confirmCompletion()
- Payment released securely

Auto Release Mechanism:

- If no response → auto-release after 3 days

Cancellation Rules:

- Job cancelled → refund to client
- No reputation impact

Security:

- Uses escrow + deadlines to prevent fraud

Reputation System

Theoretical Features

1. Stake-Based Identity System (Sybil Resistance)

Implementation:

- The contract requires a MIN_STAKE of 0.05 ETH (~₹11,500) to be deposited before any rating can be submitted

Justification:

- This creates an Identity Tax
- In a decentralized world, creating accounts is free
- By adding a financial barrier, we ensure every review is backed by "skin in the game"
- Makes it prohibitively expensive to create fake accounts for "bad-mouthing" or "ballot-stuffing"

2. Mutual Accountability (The Dual-Token Model)

Implementation:

- Reputation tokens are permission-based
- Exactly two unique tokens are issued only upon job completion (confirmCompletion)

Justification:

- Provides On-Chain Proof of Service
- You cannot leave a review unless a transaction was successfully completed
- Ensures a balanced marketplace where both parties are held to professional standards

3. The "Blind Ballot" (Anti-Retaliation Logic)

Implementation:

- A "Holding Vault" mechanism
- Scores are submitted but not applied to the public reputation until:
  - both parties have voted, OR
  - a 7-day timeout occurs

Justification:

- Solves the "Hostage Rating" problem
- Users can be 100% honest without fearing a "revenge" 1-star rating
- Scores are revealed and applied simultaneously

4. Fixed-Point Precision Scaling (UX Control)

Implementation:

- The contract multiplies the final average by 100 before division

Justification:

- Solidity does not handle decimals
- This scaling allows the UI to display a clean 1.00 to 5.00 rating
- Provides high-resolution trust data (e.g., 4.85) without rounding errors

5. Future Scope: Feedback Incentives (The Discount Model)

Implementation:

- While not currently in the core escrow code, the system is designed to trigger a Service Fee Rebate on the user's next job

Justification:

- By scaling a future discount based on the review's "Weight", we turn reputation into a financial asset
- Creates a Loyalty Loop, rewarding users for high-quality, fast, and significant feedback

Formula Flow
(The Mathematical Engine)

To arrive at the final rating, the system follows a logical progression from individual job metrics to a global trust score.

Step 1: Calculate Individual Review Weight (w)

- The "gravity" of a single review depends on:
  - the job's value
  - the reviewer's responsiveness

$$amountWeight = \frac{\max(\text{JobAmount}, 1\text{ Finney})}{1\text{ Finney}}$$

$$speedWeight = \max(7 - \text{DaysToReview}, 1)$$

$$w = amountWeight \times speedWeight$$

Step 2: Update the Reputation Ingredients

Weighted Score Sum (WSS): The total "points" earned

- The contract does not store the average
- It stores the "Weighted Sums" to maintain accuracy over time

Step 2: Update the Reputation Ingredients

Weighted Score Sum (WSS): The total "points" earned

$$WSS_{new} = WSS_{old} + (\text{Score} \times w)$$

Weight Sum (WS): The total "power" of all reviews combined

$$WS_{new} = WS_{old} + w$$

Step 3: Calculate the Public Rating (R)

$$R = \frac{WSS \times 100}{WS}$$

Result Interpretation: An integer between 100 and 500.Example: A result of 485 translates to a 4.85-star rating on the platform UI.

Gas Optimization

- **Storage Efficiency**
  - Old Contract: Used uint256 for all variables (IDs, prices, timestamps), consuming a full 32-byte slot each and increasing storage costs.
  - New Contract: Applied Tight Struct Packing using smaller types (uint32 IDs, uint88 prices, uint64 timestamps), fitting multiple values into one 32-byte slot.
  - Result: Reduced Job storage from 7 slots to 3 slots, cutting user gas fees by ~60%.

- **Increment Logic**
  - Old Contract: Used jobCount = jobCount + 1;, adding extra overhead.
  - New Contract: Uses prefix increment ++jobCount;.
  - Result: Slight gas savings per transaction

- ## Error Handling (Custom Errors)

- **Old Contract:** Used require(..., "long error string"), increasing on-chain storage and revert costs.
- **New Contract:** Switched to Custom Errors (error InsufficientStake()), using a 4-byte selector.
- **Result:** Lowered deployment gas and revert execution gas.

- ## Security & Compiler Precision

- **Old Contract:** Used floating pragma ^0.8.20, allowing version inconsistency.
- **New Contract:** Pinned pragma to 0.8.28.
- **Result:** Exact tested compiler, latest patches, audit-ready.

- ## Memory Management

- Old Contract: Re-read storage variables multiple times per function.
- New Contract: Used storage caching (Job storage j = jobs[jobId]).
- Result: Fewer SLOADs, lower gas costs.

Gas Optimization Comparison

Method	old gas	New gas	Comparison
hireFreelancer	151K	85K	↓ ~43.6%
submitFeedback	168K	117k	↓ ~30.3%
confirmCompletion	312k	257k	↓ ~17.5%
submitWork	95k	59k	↓ ~37.8%
offerService	113k	91k	↓ ~19.2%
cancelJob	63K	47k	↓ ~24.9%

Optimized Gas Report

Solidity and Network Configuration					
Solidity: 0.8.28	Optim: false	Runs: 200	viaIR: false	Block: 60,000,000 gas	
Methods					
Contracts / Methods	Min	Max	Avg	# calls	usd (avg)
FreelanceEscrow					
autoRelease	-	-	258,097	9	-
cancelJob	46,743	49,499	47,833	16	-
clearWork	-	-	27,955	4	-
confirmCompletion	-	-	257,852	37	-
depositStake	28,153	45,253	44,846	42	-
finalizeReview	33,094	119,385	61,923	9	-
hireFreelancer	-	-	85,537	66	-
offerService	74,604	91,704	91,466	72	-
submitFeedback	41,301	205,814	117,221	39	-
submitWork	-	-	59,175	50	-
Deployments				% of limit	
FreelanceEscrow	-	-	3,784,268	6.3 %	-
Key					
○ Execution gas for this method does not include intrinsic gas overhead					
△ Cost was non-zero but below the precision setting for the currency display (see options)					
Toolchain: hardhat					

Unoptimized Gas Report

Solidity and Network Configuration					
Solidity: 0.8.28	Optim: false	Runs: 200	viaIR: false	Block: 60,000,000 gas	
Methods					
Contracts / Methods	Min	Max	Avg	# calls	usd (avg)
FreelanceEscrow					
autoRelease	-	-	314,895	9	-
cancelJob	52,492	72,063	63,678	15	-
clearWork	-	-	29,760	4	-
confirmCompletion	-	-	312,733	29	-
depositStake	28,251	45,351	44,901	38	-
finalizeReview	33,135	127,274	64,580	9	-
hireFreelancer	-	-	151,684	46	-
offerService	96,390	113,490	113,147	50	-
submitFeedback	86,630	258,799	168,183	38	-
submitWork	-	-	95,074	37	-
Deployments				% of limit	
FreelanceEscrow	-	-	3,814,439	6.4 %	-
Key					
○ Execution gas for this method does not include intrinsic gas overhead					
△ Cost was non-zero but below the precision setting for the currency display (see options)					
Toolchain: hardhat					

Debugging tools

Slither:

- Static analysis on Solidity source code
- Detects:
  - Reentrancy vulnerabilities
  - Access control issues
  - Uninitialized variables
  - Gas inefficiencies
- Very fast → ideal for early-stage checks

Mythril:

- Uses symbolic execution on compiled bytecode
- Explores multiple execution paths
- Detects:
  - Reentrancy attacks
  - Integer overflow / underflow
  - Logical flaws in contract flow
- Provides deep runtime-level insights

Our contract showed no critical vulnerabilities in either tool.

Tech Stack

- Blockchain Layer: **Solidity**
- Development Framework: **Hardhat v2**
- Blockchain Interaction: **Ethers.js**
- Frontend Layer: **React.js**
- Off-Chain Storage: **IPFS** (Future Implementation)
- Wallet Integration: **MetaMask**
- Smart Contract Security: **OpenZeppelin**
- Programming Languages: **Solidity, JavaScript**

Improvements Made After Presentation

Discount Token:

- Implemented discount tokens as a proof of concept.
- Users (clients/freelancers) receive tokens for submitting feedback.
  - Token value depends on:
  - How quickly the review is submitted
- The size of the job/payment
- Currently, the tokens hold no monetary value, but could gain on-chain utility through a future slashing-based mechanism.

Email for Communication:

- Both client and freelancer now submit their email id's in their profile

Cancellation Fees:

- Added a cancellation_fee attribute to the Job struct.
- The client and freelancer mutually agree on the cancellation fee before the job begins.
- If the client cancels the job after work submission, the fee is paid to the freelancer.
- This prevents clients from misusing submitted work and cancelling the job without compensation, while maintaining fairness for both parties.

Gas Optimization Updated

Gas optimization report has been updated and uploaded under reports in GitHub

Future Improvements

Discount Token Incentives

Implemented as a proof of concept so far
Can be improved with slashing

Fraud Detection & Slashing

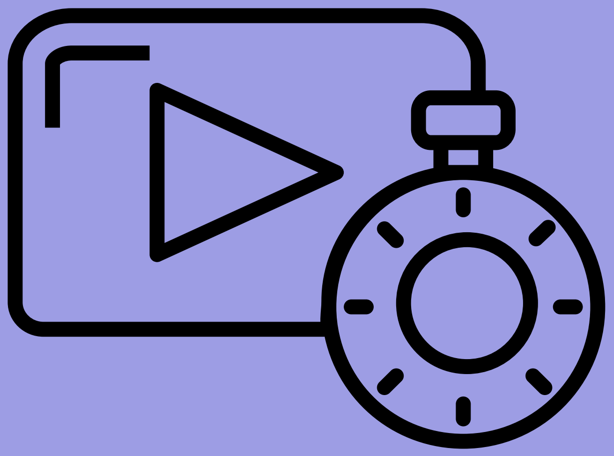
Use pattern recognition to detect misuse and penalize malicious users

Decentralized Dispute Resolution

Resolve conflicts without a central authority

Decentralized Chat (Off-Chain Consensus)

Enable secure communication between client and freelancer for agreement before on-chain actions



Demo Video

- https://drive.google.com/file/d/1TpGnNTOWfoc1PsrbHJCbasG47uMAEqqE/view?usp=drive_link
- https://drive.google.com/file/d/1I-LEgkSEOIg9RbfLtAG0BlwCZYusNwd6/view?usp=drive_link

Thank You

LedgerLogic